

Can a General-Purpose LLM Write the Other Voice? Instruction-Tuned Accompaniment Generation in Bar-Indexed ABC Notation

Amit Cohen

Tel Aviv University
amitcohen8@mail.tau.ac.il

Shai Rotman

Tel Aviv University
shairotman@mail.tau.ac.il

Yamit Kfir

Tel Aviv University
yamitkfir@mail.tau.ac.il

Hila Etziony

Tel Aviv University
hilaetziony@mail.tau.ac.il

Abstract

We ask whether a general-purpose large language model can be instruction-tuned to write an accompaniment that follows a given melody bar by bar. We write music as text in ABC notation, give the model the complete melody, and train it to write a second voice, using explicit tags that mark each bar’s voice and number so that matching bars can be lined up. We fine-tune Llama-3-8B with QLoRA on 3,012 training examples (251 pieces, each in 12 keys). On 384 test examples (32 held-out pieces in 12 keys), the base model never produces the requested voice: it continues the melody in 97% of cases, and even when its output is relabelled as the second voice, only 63% compiles cleanly and none follows the melody’s bars. After tuning, 100% of outputs are compiler-valid two-voice scores that never repeat the melody, and in 99.7% the accompaniment has exactly one bar for each melody bar, in the same order. Against the relabelled baseline, Fréchet Audio Distance to the human reference drops from 0.49 to 0.011 and note-sequence similarity to the human accompaniment (the proportion of notes that match, in order) rises from 12% to 35%. Exact agreement with the reference accompaniment remains moderate, so we claim that the model learns the task’s structure and style, not that it matches human harmonisation.

1 Introduction

Composers often start from a melody and then write the parts that accompany it. A tool that takes a finished melody and writes a matching accompaniment, one that fits the melody moment by moment, would support this way of working. Large language models are a natural candidate, because music can be written as text. Existing music language models, however, compose whole pieces from scratch rather than completing a part that is already written.

The most common text format for music is ABC notation, in which notes are written as letters and a

vertical line (|) separates the bars, the short, equal units of time into which a piece is divided. When a piece has several voices, standard ABC notation writes them one after another, so passages that are played at the same time end up hundreds of tokens apart, and language models struggle to keep the voices aligned. MuPT (Qu et al., 2024) introduced Synchronized Multi-Track ABC (SMT-ABC), which interleaves the voices bar by bar so that simultaneous bars sit next to each other; MuPT and NotaGen (Wang et al., 2025) train on such interleaved text and generate all voices of a piece together. ChatMusician (Yuan et al., 2024), an instruction-tuned music model, harmonises a melody only by writing chord names (such as C or G7) above it. None of these systems is given one complete part and asked to write another (§2). We therefore ask:

Can a general-purpose, large language model be instruction-tuned to generate a structurally synchronized, complementary musical accompaniment track when conditioned on a primary melody track?

We study this question with chorales: hymn tunes harmonised for four voices, from highest to lowest: soprano, alto, tenor and bass. Almost all of ours are Bach-style chorales from DeepChoir (Wu and Sun, 2023; Seeker38, 2024). The soprano carries the melody and forms the prompt; the alto is the accompaniment the model must write (Figure 1). Since the model must see the whole melody before it writes, the two voices cannot be interleaved. Instead, every bar is tagged with its voice and bar number, such as [V:1 B3] and [V:2 B3], so the model can line up each accompaniment bar with the melody bar of the same number. We call this format bar-indexed ABC: it keeps SMT-ABC’s principle that corresponding bars must be linked explicitly, but links them by number rather than by position. We fine-tune the general-purpose

Prompt (instruction and melody) ### Instruction: Generate a complementary harmonic accompaniment track for the following melody. ### Primary Melody (SMT-ABC): X:1 L:1/8 M:4/4 K:C [V:1 B1] z3 E [V:1 B2] A B c d [V:1 B3] B3/2 A/ G G . . .
Target (accompaniment the model must write) ### Accompaniment Track (SMT-ABC): [V:2 B1] z3 C/D/ [V:2 B2] E E E D [V:2 B3] D/E/ F E E . . .

Figure 1: A shortened example from the test set. The header gives the tune’s reference number (X), default note length (L), metre (M) and key (K); letters are notes, numbers and slashes lengthen or shorten them, and z is a rest. Each tag [V:v Bb] gives the voice and bar number, so bar B2 of the accompaniment must fit bar B2 of the melody. The headings read “SMT-ABC”, the format our representation is adapted from; the format itself is bar-indexed ABC (Figure 3, §2).

Llama-3-8B model (Grattafiori et al., 2024) on 3,012 training examples (251 chorales, each in 12 keys) with QLoRA (Detrmers et al., 2023), a memory-efficient fine-tuning method that fits on the single GPU available to us.

Before fine-tuning, the model does not write an accompaniment at all: it continues the melody. Even when we relabel its continuation as the accompaniment, only 63% of the outputs are valid music files and none lines up with the melody’s bars. After fine-tuning, every output is a valid two-voice score, and in 383 of the 384 test cases the accompaniment has exactly one bar for each melody bar, numbered in the same order (Figure 2). Automatic comparisons with the human accompaniments also favour the fine-tuned model. When the scores are rendered to audio, their Fréchet Audio Distance (FAD), a standard measure of how far generated audio is from real audio, is 0.011 against 0.49 for the relabelled baseline, and the proportion of notes that match the human accompaniment in order rises from 12% to 35% (Table 1).

Our contributions are: (i) a dataset of 314 chorale melody–accompaniment pairs in bar-indexed ABC, built from a heavily duplicated public corpus, with transposition to 12 keys applied only after the train/test split so that no piece appears in two splits; (ii) evidence that fine-tuning a general-purpose 8B model on 3,012 examples (251 pieces) is enough to learn the format and bar structure of this task; and

(iii) an evaluation against the untuned model, both as-is and with its output relabelled as the accompaniment, which also shows that three automatic metrics are unreliable for this task: two harmony scores between melody and accompaniment that rate a copied melody highly, and a time-signature check that accepts incomplete bars (Appendix A).

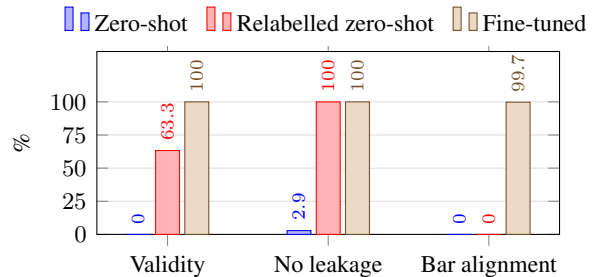


Figure 2: **The untuned model does not write the accompaniment, even after relabelling; the fine-tuned model does, in the melody’s bar structure.** Validity requires error-free compilation, a readable MIDI file, a V: 2 part and two non-empty tracks; *no leakage* means the output does not repeat the melody voice; *bar alignment* means the accompaniment has exactly one bar for each melody bar, in the same order; $n = 384$ (§4).

2 Background and Literature Review

2.1 Music as text and the bar-alignment problem

Many symbolic music systems encode a score in the style of MIDI, the standard digital format for recorded performances: a sequence of events such as “note 60 starts”, “wait 120 ms” and “note 60 ends” (Qu et al., 2024; Wang et al., 2025). For a general-purpose language model this vocabulary is foreign, and a piece becomes a very long sequence of events. ABC notation avoids both problems. It is plain text: a header gives the metre (beats per bar), key and one V: field per voice, and the body writes each note as a letter (C D E), with a number for its duration and | marking the end of each bar. A bar (or measure) is a short unit of musical time of fixed length, such as four beats; a score is a sequence of bars, and simultaneous voices share the same bar boundaries. Yuan et al. (2024) show that continued pre-training and instruction tuning on ABC text produce a capable symbolic music model without any audio-specific components; by contrast, GPT-4 with prompting alone scores 25.6% on the reasoning part of their four-option music-theory benchmark, close to the 25% chance level. Their work is limited to a single line of music: its

melody-harmonisation task outputs chord names, not a notated second voice.

ABC can express several voices, but standard ABC writes them one after another: every bar of $V:1$, then every bar of $V:2$. In a tune of B bars, bar k of voice 1 and bar k of voice 2 are separated by the $B - k$ remaining bars of voice 1 and the $k - 1$ preceding bars of voice 2, $B - 1$ bars in total for every k . Notes that sound together are therefore almost a whole piece apart in the text; in our data, bar k of the two voices is on average 282 tokens apart (median 268, range 104–606). [Qu et al. \(2024, §3.2\)](#) report that models “struggle with bar alignment when dealing with multiple tracks” and attribute this to the distance between bars that share an index.

Their Synchronized Multi-Track ABC (SMT-ABC) format ([Qu et al., 2024](#)) reorders the text: for each bar number k , the k -th bar of every voice is written in voice order, and each such group is enclosed in the delimiter `<|>`, so bars that are played together are adjacent (Figure 3, middle). NotaGen’s “interleaved ABC” expresses the same idea with an inline `[V:n]` voice tag and a per-line `[r:]` bar counter, and [Wang et al. \(2025\)](#) describe the two formats as equivalent. Interleaving is not possible in our task, because the prompt must contain the complete melody and the output only the accompaniment. We therefore adapt SMT-ABC: we keep standard sequential ABC and carry its synchronisation through an inline tag, in the style of NotaGen’s, that names the voice and also numbers the bar (Figure 3, bottom). Like SMT-ABC, this links bars that are played together, but by matching bar numbers rather than by placing them side by side (§3).

Automatic harmonisation of chorales, that is, writing the other voices for a given chorale melody, was studied with specialised neural models well before general-purpose language models. DeepChoir ([Wu and Sun, 2023](#)), for example, generates four-part chorale harmonisations with a dedicated architecture, and its data is the source of most multi-voice scores in our corpus. We study a narrower version of this task, writing one voice from the melody alone, and ask whether a general-purpose pre-trained text model can learn it through instruction tuning.

Such tuning does not require retraining the whole model. LoRA trains a small low-rank update to selected weight matrices while the pre-trained weights stay frozen ([Hu et al., 2022](#)), and QLoRA

Standard ABC (voices in sequence)	
V:1	z3 E/F/ G A G C
V:2	z6 C2 C2 C2 C2 CD
SMT-ABC (MuPT; bars interleaved)	
< >	z3 E/F/ z6 C2 < >
< >	G A G C C2 C2 C2 CD < >
Bar-indexed ABC (ours; adapted from SMT-ABC)	
[V:1 B1]	z3 E/F/ [V:1 B2] G A G C
[V:2 B1]	z6 C2 [V:2 B2] C2 C2 C2 CD

Figure 3: The same two bars of a two-voice piece in three text layouts (example adapted from [Qu et al., 2024](#)). In standard ABC, bar 1 of the two voices is separated by the rest of voice 1. SMT-ABC places each pair of simultaneous bars together. Our format keeps the sequential order, so the melody can form the prompt and the accompaniment the output, and links the bars through the numbers B1, B2.

additionally stores the frozen weights in 4-bit precision ([Dettmers et al., 2023](#)), so an 8B model ([Grattafiori et al., 2024](#)) can be fine-tuned on a single GPU.

2.2 What prior work leaves open

MuPT motivates SMT-ABC by bar misalignment but reports no alignment metric. It supports the format with a training-loss comparison in which most of the improvement comes from a longer context window, while SMT-ABC itself lowers the loss only “slightly”. [Wang et al. \(2025\)](#) add a Bar Alignment Error metric and report the best quality on interleaved ABC. Their “text-conditional control”, however, selects from three fixed labels (period, composer and instrumentation) rather than following free-text instructions. They also attribute MuPT’s higher rate of note-duration errors to its tokenizer. Byte-pair encoding (BPE) splits text into frequent multi-character pieces, so a note letter and its duration, such as C2, can become a single token, whereas predicting one character at a time keeps them apart. Llama-3 also uses a BPE tokenizer, so our model may be exposed to the same kind of error.

Neither system performs conditional in-filling. MuPT conditions on a first bar and generates all voices together, and NotaGen generates a whole piece from its opening bar and a label. Neither is given a complete voice and asked to write the missing one. Prior work thus establishes both a bar-synchronised text representation and the ability of language models to read ABC, but it has not tested whether a general-purpose model can learn

Algorithm 1 End-to-end pipeline

```
1: Data: select four-voice chorales; keep soprano V:1 and  
   alto V:2; tag every bar with voice and bar number  
2:  $x \leftarrow \text{instruction} \oplus \text{V:1}; y \leftarrow \text{V:2}$   
3: remove duplicate prompts; split 80/10/10 (seed 42);  
   transpose  $-5..+6$  within each split  
4: Train:  $M_\theta \leftarrow \text{QLORA}(M_0, \text{train})$   
5: Generate:  $\hat{y}_{zs} \leftarrow M_0(x), \hat{y}_{rl} \leftarrow M_\theta(x)$   $\triangleright$  greedy  
6:  $\hat{y}_{rl} \leftarrow \hat{y}_{zs}$  with V:1 renamed V:2  
7: for all  $\hat{y} \in \{\hat{y}_{zs}, \hat{y}_{rl}, \hat{y}_{rl}, y\}$  do  
8:   compile prompt  $\oplus \hat{y}$  to MIDI; check validity and bars  
9:   if valid: compare the accompaniment with  $y$ ; render  
   audio  
10: end for
```

to fill in one voice from another through parameter-efficient instruction tuning on a single GPU, rather than through pre-training a music-specific model (Qu et al., 2024; Wang et al., 2025; Yuan et al., 2024). This paper addresses that question.

3 Methodology

3.1 Task Formulation

We treat accompaniment generation as conditional in-filling of a text score. The prompt contains the instruction, the ABC header and the complete melody (the soprano, voice V:1); the target is the alto voice V:2, which serves as the accompaniment once tenor and bass are removed. Every bar begins with a tag giving its voice and bar number, and z denotes a rest, a silence (Figure 1). The prompt and target carry fixed section headings, “Primary Melody” and “Accompaniment Track”, labelled “SMT-ABC” after the format ours is adapted from. All examples use the same instruction, because the corpus has no instrument or style annotations; the task therefore tests whether the model learns to write a matching accompaniment, not whether it follows varied instructions.

Bar-indexed ABC. Our format is a variant of SMT-ABC and NotaGen’s interleaved ABC: standard sequential ABC with a tag at the start of every bar (Figure 3). Because the whole melody comes first, [V:1 B3] and [V:2 B3] are a full melody apart, and their position in the text says nothing about which bars belong together. The model must instead reproduce the melody’s bar numbers, in order, in its own tags. The tags follow NotaGen’s inline [V:] voice markers (Wang et al., 2025), extended with a bar number, and they let us measure alignment directly from the text.

3.2 Dataset Construction

Algorithm 1 summarises the pipeline. We build the dataset from Seeker38/music_abc_notation, which contains 383,940 ABC scores in instruction format and is the only candidate corpus with multi-voice ABC. The alternative, sander-wood/wikimusictext, holds 1,010 lead sheets (a single melody line whose harmony appears only as chord names) (Wu et al., 2023), so it has no second voice to predict.

Filtering. 33,503 rows contain both a V:1 and a V:2 voice. All of them are four-voice chorales (V:1–V:4), almost all Bach-style chorales from DeepChoir. We keep V:1 and V:2 and discard V:3 and V:4, so the target alto was originally written to sound with a tenor and bass that the task removes. The preprocessing script removes formatting lines (I:linebreak, %score), rejects scores with unbalanced brackets, and adds the bar tags.

Deduplication and splitting. The 33,503 rows contain only 319 distinct scores: 298 appear 100 times, 17 appear 200 times, one appears 300 times and three appear once. Of these, 314 pass preprocessing. We remove duplicate prompts *before* splitting and then split the 314 scores 80/10/10 with seed=42 into 251 training, 31 validation and 32 test pieces. No melody or accompaniment appears exactly in more than one split. Deduplication compares the full prompt text, including the header. After removing headers, two melodies still occur twice each, all four copies in the training set and each with a different accompaniment, so they are two harmonisations of the same tune rather than duplicates.

Augmentation. Within each split, we transpose every piece with `abc2abc -t`, shifting all its notes by the same interval, from -5 to $+6$ semitones (the smallest pitch step). Transposition changes the key but not the music. Each piece therefore appears in 12 keys, including the original, which gives 3,012 training, 372 validation and 384 test examples. Because transposition happens after splitting, no piece reaches two splits in any key. The 12 versions share one harmonic and rhythmic structure, so the test set contains 32 independent pieces. The longest examples are 1,249, 1,277 and 1,054 tokens for training, validation and test, all within the 1,536-token training limit; the longest test prompt is 548 tokens.

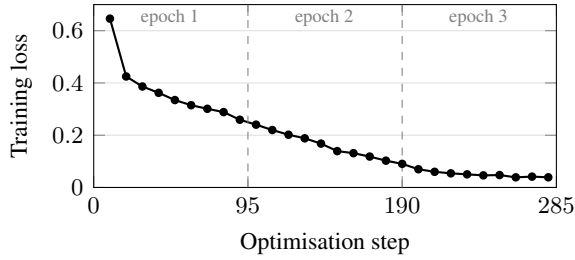


Figure 4: Training loss of the fine-tuning run (3,012 examples, effective batch 32). Token-level accuracy on the training data rises from 82.4% to 98.5% over the same steps.

3.3 Model and Training Setup

We fine-tune meta-llama/Meta-Llama-3-8B, the pre-trained checkpoint without instruction tuning, with QLoRA. During training the frozen weights are stored in NF4, a 4-bit number format designed for neural-network weights, with double quantisation (the per-block scaling constants are also compressed), and computation runs in the 16-bit bfloat16 format. LoRA adds small trainable matrices next to selected weight matrices of the model. We add them to all seven weight matrices in each transformer layer: the four projections of the attention block (query, key, value and output) and the three of the feed-forward (MLP) block. Their rank $r=16$ sets their size, $\alpha=32$ scales their contribution, and dropout 0.05 randomly drops 5% of their inputs during training to reduce overfitting. Each training example is the prompt followed by the target, and the loss is computed over the whole sequence. We use `paged_adamw_8bit`, a maximum sequence length of 1,536 and gradient checkpointing, with learning rate 3×10^{-4} , linear decay without warmup, per-device batch 1 with 32 gradient-accumulation steps (effective batch 32), 3 epochs (285 optimisation steps) and seed=42. Training ran on one GPU of the university’s shared cluster and took 13 hours 29 minutes. Figure 4 shows the training loss, which falls from 0.646 to 0.039 over the 285 steps while token-level accuracy on the training data rises from 82.4% to 98.5%. Both level off during the third epoch; the run did not record validation loss.

Hyperparameters. We compared 12 configurations (learning rate $\{1, 2, 3\} \times 10^{-4}$, 3 or 5 epochs, effective batch 24 or 32) by validation loss after training on 100 examples (Appendix B). The search was run on an earlier version of our dataset, built from the same corpus, in which the target was the

chord names written above a single-line melody rather than a second voice. It favoured learning rate 3×10^{-4} , five epochs and batch 24, but losses were close across configurations (0.594–0.621 at 3×10^{-4}). A three-epoch run on the full training set already required 13.5 hours; five epochs would need about 22.5 hours, close to the 24-hour limit of our cluster jobs, which can also be preempted. We therefore trained for three epochs at the best learning rate, trading some convergence for a run that could finish reliably.

Inference. Both models load the same NF4 weights without double quantisation and decode greedily (`do_sample=False`), so generation is deterministic. The untuned model receives up to 1,100 prompt tokens and may generate 1,100 new tokens; the fine-tuned model receives up to 2,048 and may generate 2,000. We release the code, data splits, model outputs and the trained adapter at https://github.com/AtomBonk/NLP_FINAL.

3.4 Evaluation

Baselines. The *zero-shot* baseline is the same base model without fine-tuning, given the identical prompt and decoding settings. It shows what the pre-trained model does with the instruction before any task-specific training. Because the base model writes its continuation under the melody’s label V:1, a labelling mistake alone might explain its failure. We therefore also score a *relabelled zero-shot* baseline, in which every V:1 tag in its output is renamed V:2 and the result is scored with the same metrics. This gives the base model the most favourable reading of its output and doubles as a simple melody-copying baseline. We also score the human accompaniment against itself: metrics that compare with the reference should then reach their best value, which confirms that the evaluation code works.

Structural metrics. Each output is appended to its prompt and compiled to MIDI with `abc2midi` (Allwright and Methven, 2024). An output is *valid* if the compiler finishes without error, the MIDI file can be read, the output contains a [V:2] tag, and the MIDI file has at least two non-empty tracks. *Melody leakage* is the share of outputs containing a [V:1] tag, that is, outputs in which the model writes the melody voice again; *missing accompaniment* is the share without a [V:2] tag or without a second non-empty track. *Bar alignment* is the share of outputs with exactly as many bars as the

reference, and *bar-count deviation* is the mean absolute difference in bar count. For the fine-tuned model we also compared the sequence of bar numbers in the tags with the prompt, which gives the same result.

Musical metrics. For valid outputs we isolate the accompaniment track and compare it with the reference using `music21` (Cuthbert and Ariza, 2010) and `MusPy` (Dong et al., 2020). *Scale consistency* is the share of notes that belong to the best-fitting major or minor scale, the seven-note set on which a key is built. *Key match* is the share of outputs whose estimated key, its home note (tonic) and whether it is major or minor (mode), equals that of the reference accompaniment. *Note-sequence similarity* compares the sequences of note names (C, D, E, ...) in the generated and reference accompaniments with Python’s `SequenceMatcher`: the score is $2M/T$, where M is the number of notes in matching stretches found in the same order and T is the total number of notes in both sequences, so identical sequences score 100%. Key match and note-sequence similarity measure agreement with one human accompaniment; a different but valid accompaniment scores lower. *Fréchet Audio Distance* (FAD) (Kilgour et al., 2019) measures how different two collections of audio are. We render every reference and generated score to audio with the `FluidSynth` synthesiser and pass it through `VGGish`, a neural network trained to classify sounds (Hershey et al., 2017), which summarises each segment of about one second as a vector of 128 numbers. FAD fits a Gaussian distribution to the vectors of each collection and measures the distance between the two distributions; lower means the generated audio is statistically closer to the reference audio. Both renderings include the melody, so differences come from the accompaniment. Outputs that fail to compile receive a score of 0 on the musical metrics. Appendix A lists three further metrics that we implemented but excluded: two harmony scores between melody and accompaniment that rate a copied melody highly, and a time-signature check that accepts incomplete bars.

4 Results and Discussion

Table 1 reports all metrics on the 384 test examples: 32 held-out pieces, each in 12 keys. Because the 12 versions of a piece share their musical structure, the results describe 32 independent pieces.

Metric	Ref.	Zero	Relab.	Tuned
<i>Structure</i>				
Validity	100.00	0.00	63.28	100.00
Melody leakage ↓	0.00	97.14	0.00	0.00
Missing accomp. ↓	0.00	100.00	8.59	0.00
Bar alignment	100.00	0.00	0.00	99.74
Bar-count dev. ↓	0.00	51.54	51.54	0.0026
<i>Musical</i>				
Scale consistency	92.49	–	89.88	95.72
Key match	100.00	–	32.29	34.90
Note-sequence sim.	100.00	–	12.12	34.81
FAD ↓	0.0000	–	0.4918	0.0111

Table 1: Results on 384 test examples (32 pieces $\times$ 12 keys). Values are percentages except bar-count deviation (bars) and FAD; ↓ = lower is better. **Ref.** scores the human accompaniment against itself; **Zero** is the untuned model; **Relab.** is its output with V:1 renamed V:2. Zero-shot outputs contain no accompaniment, so musical metrics are undefined. Musical metrics are computed for every output that yields a readable two-track MIDI file (351 relabelled outputs) and are 0 for the rest.

Sanity check. Before full training, we fine-tuned the model on eight training examples for 30 epochs at learning rate 2×10^{-4} . The training loss fell from 1.102 to 0.006 and token-level accuracy rose from 74.8% to 99.7%, confirming that the training pipeline runs correctly. This tests the code, not generalisation.

4.1 The untuned model continues the melody

The zero-shot model never writes the accompaniment. None of its 384 outputs contains a V:2 tag; 373 continue the melody under V:1, and the remaining 11 are empty. In 368 cases the continuation starts at exactly the bar after the prompt’s last bar, for example:

```
| [V:1 B27] G | [V:1 B28] Ac c/c/c AF
F/F/F | [V:1 B29] DFAc BAGF | . . .
```

Every non-empty output uses the full 1,100-token generation budget, more than twice the length of the longest reference accompaniment (523 tokens), without switching to the requested voice. The prompts are not truncated: the longest is 548 tokens, below the 1,100-token input limit. The base model thus treats the prompt as text to be continued rather than as an instruction.

Relabelling this output as the accompaniment does not rescue it. Only 63.28% of the relabelled outputs compile without error, none has the reference’s number of bars, and 367 of 384 are longer than the reference (mean absolute difference 51.5

bars). Their note-sequence similarity to the human accompaniment is 12.12% and their FAD is 0.4918.

4.2 Fine-tuning learns the task format and bar structure

After fine-tuning, all 384 outputs are valid: each contains a V:2 part, none contains V:1, each compiles without error, and each produces a two-track MIDI file. In 383 of the 384 outputs (99.74%), the accompaniment has exactly one bar for each melody bar, tagged with the same bar numbers in the same order; the remaining output omits the final bar, giving a mean bar-count deviation of 0.0026 bars against 51.54 for the zero-shot model. The model has therefore learned to write the requested voice in valid notation and to follow the melody’s bar structure. A model could pass these bar checks trivially by writing a rest in every bar. The fine-tuned model does not: none of its 5,795 accompaniment bars consists only of rests, compared with 12 of 5,796 (0.21%) in the human accompaniments.

4.3 Musical quality

When the scores are rendered to audio, the fine-tuned outputs are much closer to the human accompaniments than the relabelled baseline is: their FAD is 0.0111, against 0.4918. Their scale consistency is 95.72%, similar to the human accompaniments’ 92.49%, so almost all generated notes belong to a single major or minor scale, as in the originals. These are automatic measures; we did not run a listening test.

Note-sequence similarity to the human accompaniment is 34.81%, almost three times the relabelled baseline’s 12.12%. To check that this is not an accident of the small test set, we estimated its uncertainty by bootstrapping: we drew 32 test pieces at random with replacement, recomputed the average, and repeated this 10,000 times. The middle 95% of these averages lies between 32.6% and 37.2%. To show what an unrelated accompaniment would score, we also compared each fine-tuned output with the human accompaniment of a *different* test piece, which gives 19.7%. The model’s outputs therefore match the accompaniment written for their own melody clearly better than chance.

The estimated key of the output agrees with that of the human accompaniment in 34.90% of cases, only slightly more than for the relabelled baseline (32.29%). Estimating a key from a single voice is difficult, because one line of a chorale may contain too few notes to identify its key, so this metric is

noisy.

Both key match and note-sequence similarity compare the output with the one human accompaniment in the dataset. A melody can be accompanied well in many different ways, so an output that differs from the human part is not necessarily wrong, and these metrics may underestimate its quality. Judging that directly would require human listeners.

4.4 Limitations

The training set contains 3,012 examples but only 251 distinct pieces, and the test set 384 examples from 32 pieces. All pieces are Western four-part chorales, and each reference accompaniment was originally an alto part written to sound with tenor and bass parts that the task removes. We trained one model with one seed and one instruction wording; greedy decoding removes sampling variation but not these sources of variance. We compare against the untuned model rather than a published music-generation system, and our musical metrics are automatic.

5 Conclusion

We asked whether a general-purpose language model can be instruction-tuned to write an accompaniment that follows a given melody bar by bar. The untuned Llama-3-8B model never writes the requested voice and only continues the melody, and relabelling its output does not produce a usable accompaniment. After QLoRA fine-tuning on 3,012 examples (251 pieces in 12 keys), every output is a valid two-voice score, in 99.74% the accompaniment has one bar for each melody bar in the same order, and the generated accompaniments are far closer to the human reference when rendered to audio (FAD 0.0111 against 0.4918). Agreement with the particular human accompaniment remains moderate, which leaves room for improvement and for evaluation beyond exact-reference metrics. Natural next steps are a listening study, more training data and seeds, and accompaniments written for the two-voice setting.

AI Disclosure and Reflection

We used AI tools at three stages of the project. While building the pipeline, we used Anthropic’s Claude Opus 4.8, and later Claude Opus 5, to help debug our Python code and to show us how to use libraries we had not worked with before. We

used OpenAI’s ChatGPT 5.6 for help with LaTeX formatting. In the final stage, we used Claude Opus 5.5 through Claude Code to check the paper against our code, logs and data. It re-ran our analyses, flagged claims the data did not support, wrote and ran the repeated sanity check (§4), computed the bootstrap interval, and helped us rewrite much of the text for clarity. We chose the research question, designed the experiments, ran the main training and evaluation jobs, and decided which suggestions to accept.

The tools were most useful for unfamiliar syntax, routine debugging and checking many numbers quickly. They were least reliable when code looked correct but rested on a wrong assumption. Our first evaluation script, written with AI help, ignored compiler errors, measured bar alignment by counting bars only, accepted bars that were too short, and included two harmony scores that rewarded copying the melody (Appendix A). During the final review, a bar-alignment check also first missed the space that transposition adds to our bar tags, which made all 384 outputs look aligned instead of 383. We found these problems by testing each metric on cases with a known answer, such as the human accompaniment scored against itself and a copied melody, and by reading the code behind each reported number. Our main lesson is that code and text written with AI need the same checks as our own.

References

- James Allwright and Seymour Shlien Methven. 2024. abcmidi: abc2midi, abc2abc and companion utilities. <https://ifdo.ca/~seymour/runabc/top.html>.
- Michael Scott Cuthbert and Christopher Ariza. 2010. music21: A toolkit for computer-aided musicology and symbolic music data. In *International Society for Music Information Retrieval Conference (ISMIR)*, pages 637–642.
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. **QLoRA: Efficient finetuning of quantized LLMs**. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Hao-Wen Dong, Ke Chen, Julian McAuley, and Taylor Berg-Kirkpatrick. 2020. **MusPy: A toolkit for symbolic music generation**. In *International Society for Music Information Retrieval Conference (ISMIR)*.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. 2024. **The Llama 3 herd of models**. *Preprint*, arXiv:2407.21783.
- Shawn Hershey, Sourish Chaudhuri, Daniel P. W. Ellis, Jort F. Gemmeke, Aren Jansen, R. Channing Moore, Manoj Plakal, Devin Platt, Rif A. Saurous, Bryan Seybold, Malcolm Slaney, Ron J. Weiss, and Kevin Wilson. 2017. CNN architectures for large-scale audio classification. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. **LoRA: Low-rank adaptation of large language models**. In *International Conference on Learning Representations (ICLR)*.
- Kevin Kilgour, Mauricio Zuluaga, Dominik Roblek, and Matthew Sharifi. 2019. Fr chet Audio Distance: A reference-free metric for evaluating music enhancement algorithms. In *Interspeech*, pages 2350–2354.
- Xingwei Qu, Yuelin Bai, Yinghao Ma, Ziya Zhou, Ka Man Lo, Jiaheng Liu, Ruibin Yuan, Lejun Min, Xueling Liu, Tianyu Zhang, Xinrun Du, Shuyue Guo, Yiming Liang, Yizhi Li, Shangda Wu, Junting Zhou, Tianyu Zheng, Ziyang Ma, Fengze Han, and 9 others. 2024. **MuPT: A generative symbolic music pre-trained transformer**. *Preprint*, arXiv:2404.06393.
- Seeker38. 2024. **music_abc_notation**. Hugging Face Datasets.
- Yashan Wang, Shangda Wu, Jianhuai Hu, Xingjian Du, Yueqi Peng, Yongxin Huang, Shuai Fan, Xiaobing Li, Feng Yu, and Maosong Sun. 2025. **Notagen: Advancing musicality in symbolic music generation with large language model training paradigms**. *Preprint*, arXiv:2502.18008.
- Shangda Wu and Maosong Sun. 2023. **Chord-conditioned melody harmonization with controllable harmonicity**. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*.
- Shangda Wu, Dingyao Yu, Xu Tan, and Maosong Sun. 2023. **CLaMP: Contrastive language-music pre-training for cross-modal symbolic music information retrieval**. In *Proceedings of the 24th International Society for Music Information Retrieval Conference (ISMIR)*, pages 157–165, Milan, Italy.
- Ruibin Yuan, Hanfeng Lin, Yi Wang, Zeyue Tian, Shangda Wu, Tianhao Shen, Ge Zhang, Yuhang Wu, Cong Liu, Ziya Zhou, Liumeng Xue, Ziyang Ma, Qin Liu, Tianyu Zheng, Yizhi Li, Yinghao Ma, Yiming Liang, Xiaowei Chi, Ruibo Liu, and 13 others. 2024. **ChatMusician: Understanding and generating music intrinsically with LLM**. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 6252–6271, Bangkok, Thailand. Association for Computational Linguistics.

A Excluded metrics

Metric	Reason for exclusion
Time-signature compliance	Counts a bar as correct if its notes do not exceed the bar length, so bars that are too short also pass.
Melody–accompaniment consonance	Counts every moment with at most two sounding pitches as consonant, which holds almost always for two single-line voices. The relabelled melody copy scores 90.52%.
Melody–accompaniment key compatibility	A weighted score of tonic and mode agreement that ranks the relabelled melody copy (84.35%) above the human accompaniment (52.63%).

Table 2: Metrics implemented in our evaluation script but excluded from the results because they reward copying the melody or accept incorrect bars.

Two definitions of validity. Our evaluation script counts an output as valid whenever `abc2midi` produces a readable two-track MIDI file with a V:2 part, even if the compiler reports errors. Table 1 uses a stricter definition that also requires compilation without errors (§3). For the human and fine-tuned accompaniments both definitions give 100%. For the relabelled baseline, 351 of the 384 outputs (91.41%) produce such a file, but only 243 (63.28%) compile without errors. Table 1 reports the stricter 63.28%, while its musical metrics for this baseline are computed on the 351 readable files.

B Hyperparameter search

LR	Epochs	Eff. batch	Val. loss
3e-4	5	24	0.5944
2e-4	5	24	0.6002
3e-4	5	32	0.6020
2e-4	5	32	0.6120
3e-4	3	24	0.6126
1e-4	5	24	0.6170
3e-4	3	32	0.6208
2e-4	3	24	0.6239
1e-4	5	32	0.6255
2e-4	3	32	0.6308
1e-4	3	24	0.6558
1e-4	3	32	0.6928

Table 3: All 12 configurations (LoRA rank 16), ranked by validation loss after training on 100 examples and evaluating on 50. The search used the earlier version of the dataset described in §3. Longer training and higher learning rates scored better throughout; the final run uses learning rate 3×10^{-4} with 3 epochs and batch 32 because of the compute limit.